

高级语言程序设计

实验报告

南开大学 计算机科学与技术

姓名 韩硕

学号 2512752

班级 计算机卓越拔尖计划伯苓 2 班

2026 年 5 月 13 日

目录

高级语言程序设计大作业实验报告	3
一. 作业题目	3
二. 开发软件与运行环境	3
三. 课题要求	3
四. 项目总体设计	3
1. 整体流程	3
2. 类与模块划分	3
3. 游戏循环设计	4
五. 主要模块设计与实现	4
1. 状态机与界面控制	4
2. 实体状态的封包与解包算法	4
3. 基于矩阵卷积的群体伤害结算	4
4. 怪物寻路与防重叠机制	4
六. 功能测试	5
七. 打包部署与仓库提交	6
八. AI 工具使用说明	6
九. 开发中遇到的问题与解决方法	6
十. 收获与总结	6

高级语言程序设计大作业实验报告

一、作业题目

本次大作业题目为《重生之我在 nku 制霸 meme 家族》。本项目为一款基于 C++ 与 Qt 框架独立开发的 2D 俯视角生存割草游戏。

二、开发软件

本项目使用 C++ 语言和 Qt 框架完成

项目	内容
开发语言	C++
图形界面框架	Qt Widgets
开发工具	Qt Creator
编译器套件	MinGW 64-bit
运行依赖	Qt Core, Qt Gui, Qt Multimedia 模块

三、课题要求

- (1) 使用 C++ 语言完成一个图形化小程序；
- (2) 程序内容主题不限，应体现一定创新性和完整性；
- (3) 项目需提交到 Gitee 或 GitHub，并体现多次版本迭代；
- (4) 提交项目报告，说明设计思路、技术路线、算法与功能实现；
- (5) 如使用 AI 工具，需要在报告中说明工具名称、用途和使用位置。

四、项目总体设计

1、整体流程

程序启动后首先进入主菜单界面，玩家可查阅游戏规则、选择角色并进入环境选择（支持常规沉浸模式与背景透明的“桌宠模式”）。进入战斗状态后，游戏通过 QTimer 定时器以每 16ms 一次的频率刷新逻辑循环，依次完成玩家输入检测、全局 AI 矩阵寻路、碰撞检测与画面重绘。当玩家生命值降至 0 时游戏失败，当击杀数达到 150 触发最终波次并击败 Boss 后，游戏胜利。

2、类与模板划分

模块/类	主要职责	说明
Gamewindow	游戏主循环与渲染管线	负责状态机流转、界面绘制、定时器驱动、音效调度与按键事件拦截。
jvzhen	底层物理与数据基石	自定义二维矩阵结构，全局实体的坐标与状态均映射于该动态数组上。
Layer (基类)	多态架构调度中心	抽象出虚基类，规范统一的 forward 更新接口，供游戏主循环调度。
ConvLayer	二维卷积伤害引擎	继承自 Layer，负责将玩家的技能权重矩阵与当前战场矩阵进行点积运算。
MonsterAllayer	全局 AI 与体积排斥	继承自 Layer，负责扫描玩家坐标，并利用矩阵占位检测执行怪物的贪心趋附。

GameUtils	数据封包与解包结算	内置 apply_damage 等轻量级算法，利用浮点数特性分离怪物的血量与种族数据。
-----------	-----------	---

3、游戏循环设计

本项目使用 Qt 核心的 QTimer 定时器机制，定时器每隔 16ms 触发一次 gameLoop() 函数，构成 60FPS 的游戏主心跳。在每一帧中，引擎会结合有限状态机 (gameState) 分发逻辑：依次执行怪物坐标推演、CD 冷却削减、平滑插值计算以及画面的 update() 重绘请求。

五、主要模块设计与实现

1. 状态机与界面控制

游戏在 Gamewindow 中设计了极其严密的有限状态机逻辑：状态 0 处理主菜单；状态 1 负责角色选择；状态 2 执行数据预热并判定是否开启无边框透明的“桌宠模式”；状态 3 为核心战斗场景；状态 4 处理大招解锁的剧情转场。所有按钮的交互均通过计算鼠标坐标与隐形 QRect 区域的包含关系来实现，避免了大量堆砌 UI 控件带来的性能损耗。

2. 实体状态的封包与解包算法

为了优化内存与检索速度，项目在 GameUtils.h 中设计了独特的数据压缩技巧。将怪物的属性拼凑为一个 double 类型的浮点数：整数部分存储血量，小数部分存储种族编号。读取时利用 std::floor 提取血量，利用 std::round 提取种类，单次网格访问即可掌握实体的全部状态，大幅降低了大规模战斗时的计算开销。

3. 基于矩阵卷积的群体伤害结算

本项目放弃了传统的欧氏距离检测，转而在 ConvLayer 中引入了图像处理领域的卷积运算思想。将玩家的技能（如十字斩、裂风）预设为小型的权重矩阵。释放技能时，将技能矩阵覆盖在当前战场矩阵上进行双层循环的点积求和。这直接把范围伤害判定降维成了纯数学计算，在同屏数百只怪物时依然能保持丝滑的帧率。

4. 怪物寻路与防重叠机制

控制怪物移动的逻辑集中在 MonsterAILayer.h 中。为了防止数千个怪物贴图挤成一团（穿模），底层加入了严格的网格占位校验机制。在执行朝向玩家的贪心移动前，怪物必须探查目标网格的数据：若数值为 0，则允许移入并刷新坐标；若不为 0（已被占用），则在原地待命。这种底层强制互斥逻辑，从根源上实现了怪物的体积碰撞排斥效果。

六、功能测试

本项目主要采用运行测试和功能点测试的方式进行验证：

(1) 状态机流转与桌宠模式渲染测试

测试过程与输入：在状态 2 界面点击“桌宠模式”，追踪程序向系统请求窗口透明化及无边框处理的执行情况，并观察主循环进入状态 3 后的渲染表现。

预期与实际结果：底层触发 setWindowFlags 与 WA_TranslucentBackground

属性。游戏进入战斗后，背景图层完全透明化剥离，仅保留实体矩阵渲染，无黑边或画面残影，且键盘事件拦截正常。

测试通过。

(2) 底层矩阵浮点数解包精度极值测试

测试过程与输入：针对 GameUtils.h 中的解包算法，人为向矩阵注入极低血量的极端双精度测试数值（例如数值为 0.001 的 6 号 Boss 类型），验证其分离逻辑。

预期与实际结果：利用 `std::floor` 与 `std::round` 的组合解包逻辑，能够精准隔离整数部分（当前血量）与小数部分（种族标记）。在面临 IEEE 754 标准的浮点数极限时，未出现因精度丢失导致的怪物类型突变或无法判定死亡的 Bug。

测试通过。

(3) ConvLayer 卷积伤害引擎边界防崩溃测试

测试过程与输入：将玩家角色移动至地图绝对原点 (0, 0) 及极限右下角边界，连续释放最大范围 (5x5 权重矩阵) 的终极大招“裂风”，监控内存访问状态。

预期与实际结果：卷积核的双层循环逻辑能够根据地图边界变量准确计算并截断偏移量 (offsetH/ offsetW)。技能仅对合法范围内的矩阵网格执行点积运算，成功阻断了越界访问，程序未发生段错误或崩溃。

测试通过。

(4) 基于矩阵扫描的体积排斥抗压测试

测试过程与输入：屏蔽怪物生成上限，利用测试脚本瞬间生成远超当前地图总网格数量的海量实体，观察 MonsterAILayer 寻路引擎的调度表现。

预期与实际结果：所有怪物在向玩家趋附前，均严格执行了 `target_cell == 0` 的网格强一致性占位校验。当周边网格满载后，底层阻塞逻辑正常触发，怪物保持原地待命。全程无任何不同实体的坐标发生重叠穿模，且 0(1) 的校验开销保证了帧率稳定。

测试通过。

(5) 环境交互与状态冷却时序测试

测试过程与输入：在玩家满血溢出状态，以及残血状态下，分别精准点击坐标标记为 77.0 的 KFC 治愈网格。并在其进入冷却状态时进行高频恶意连点。

预期与实际结果：系统精准将屏幕像素坐标映射为矩阵坐标并命中检测区域。血量回复严格受 `playerMaxHP` 阈值限制（溢出截断）。触发后网格数值瞬间翻转为 -77.0，并开启长达 250 逻辑帧的严格冷却。冷却期间的高频点击被底层彻底免疫，时序逻辑完美闭环。

测试通过。

(6) 动态链接库剥离与纯净环境运行测试

测试过程与输入：使用 `windeploymt` 完成依赖库封包，将最终的 .zip 免安装包放置于未安装任何 Qt 环境、C++ 编译器及相关系统变量的 Windows 裸机虚拟机中解压运行。

预期与实际结果：可执行文件 (.exe) 成功调取同级目录下的动态链接库（如 `Qt5Core.dll` 等）。所有基于相对路径 (:/) 挂载的资源文件（包括 GIF 动图与 WAV 音频）均由二进制文件内存直接解码播放，实现了真正的“双击即玩”。

测试通过。

七、打包部署与仓库提交

项目源码已提交至 GitHub 与 Gitee 仓库，仓库内保留了 1.0 到 3.0 等多个版本的历史记录，清晰体现了架构从搭建到重构的完整迭代过程。为了方便助教与老师直接评阅，项目根目录新建了独立文件夹，并使用 windeployqt 工具完成了 Qt 依赖库的链接封包。最终将免安装试玩包一并上传至云端，评阅人只需下载压缩包双击.exe 即可畅玩。

GitHub 仓库链接 <https://github.com/hanshuo0307-create/c.homework>
Gitee 仓库链接 <https://gitee.com/yeweijv/c-homework>
B 站讲解视频链接 <https://www.bilibili.com/video/BV1ESRyBFMvc?t=2.7>

八、AI 工具使用说明

AI 工具	版本/类型	用途	使用位置
Gemini/ChatGPT	大语言模型	辅助梳理底层逻辑框架，优化运算符重载规则与浮点数解包精度问题。	核心代码算法层、实验报告大纲构建。
AI 生成工具	图像生成	辅助生成游戏内的部分高质量背景贴图与技能光效素材。	资源目录(resources.qrc)

核心业务逻辑（状态机流转、底层矩阵卷积判定、自写 AI 寻路排斥等）均由本人独立设计与实现。

九、开发中遇到的问题与解决方法

问题	原因分析	解决方法
海量实体渲染导致帧率骤降	传统的面向对象设计导致内存中存在过多重实体。	抛弃第三方引擎，重构成自底向上的矩阵驱动架构，并利用浮点数压缩实体状态。
高密度状态下怪物完全重叠	贪心算法导致所有怪物共用同一最优坐标。	在寻路推演逻辑中加入强一致性的网格占位锁，目标不为 0 则禁止写入。
GitHub 无法上传打包后的游戏	打包后压缩包体积超过 50MB，触发 GitHub LFS 警告。	使用 Git 命令行重置提交并直接推送，刚好控制在 100MB 硬限制以下成功上传。

十、收获与总结

通过本次《重生之我在 nku 制霸 meme 家族》大作业，我深刻体会到了“自底向上”搭建底层算法的魅力与挑战。相比于调用现成的游戏引擎，亲自手写二维矩阵物理网格并引入“卷积运算”来解决性能瓶颈，极大地锻炼了我的 C++ 核心编程能力与算法优化思维。

同时，我在项目工程化方面也积累了宝贵的经验。从版本控制（Git 迭代与冲突解决）、撰写规范的 README 文档，到最终使用命令行工具进行环境打包部署，我完整体验了一套工业级的软件开发流水线，这为我未来的专业课程学习与复杂系统开发打下了极其坚实的基础。